



SW개발/HW제작 설계서

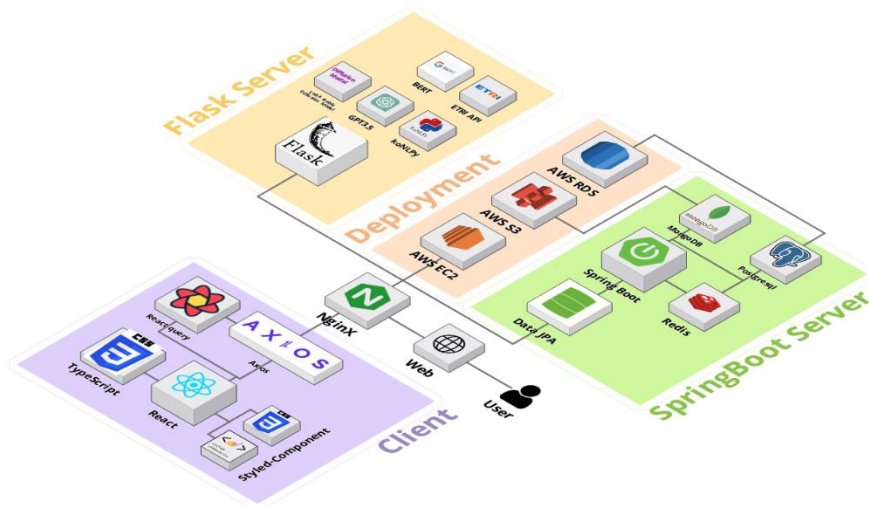
프로젝트 명 : NLP 기반 소설 IP 확장 솔루션

| 요구사항 정의서

구분	기능	설명
S/W	소셜 로그인	회원가입(구글/네이버)
		로그인
	소셜 파일 업로드	텍스트 파일로 소설을 업로드한다.
	소셜 텍스트 작성	소설을 텍스트로 작성한다.
	대본 변환 요청	소설을 비대사, 대사 텍스트 분류와 문장 간 연관 분석을 통해 대본으로 변환한다.
	대본 파일 다운로드	변환된 대본을 파일로 다운로드한다.
	대본 수정	대본을 직접 수정하여 완성도를 높인다.
	주인공 수정	상호참조 변환으로 부정확한 부분을 직접 수정하여 정확도를 높인다.
	스토리보드 생성 요청	대본에서 씬(Scene), 컷(Cut) 단위로 보드를 생성한다.
	채팅방 생성	소설 관련 질의응답을 진행할 채팅방을 생성한다.
	채팅 메시지 보내기	챗봇에게 질문을 하면 답변을 해준다.
	마이페이지	변환된 대본과 스토리 보드 확인하기 각 페이지에 6개의 스토리보드 목록 보여줌
	변환 정보 시각화	소설의 배경과 등장인물, 등장인물의 출연 비율을 마인드맵, 차트 등으로 시각화한다.

구분	기능	설명
S/W	서비스 소개	해당 서비스에 대한 안내와 설명을 보여준다.
	대본화 체험하기	비로그인 사용자도 접근 가능하다. 샘플 소설로 대본화 체험을 할 수 있다.

| 서비스 구성도 - 서비스 시나리오



Client

- 클라이언트, 스프링부트 서버 간 통신: React-query, axios를 사용한 API 통신
- 소설 데이터 전송, 반환값 확인: 소설 데이터(title, content) 전송 후 반환된 등장인물 상호참조 및 대본 데이터 확인

Spring boot Server

- 관계형 DB 사용방식 정의: Data JPA
- 클라이언트 단과의 API 통신
- 대본, 스토리보드 데이터 반환 및 DB(Postgresql, MongoDB) 저장
- 캐시 관리: Redis

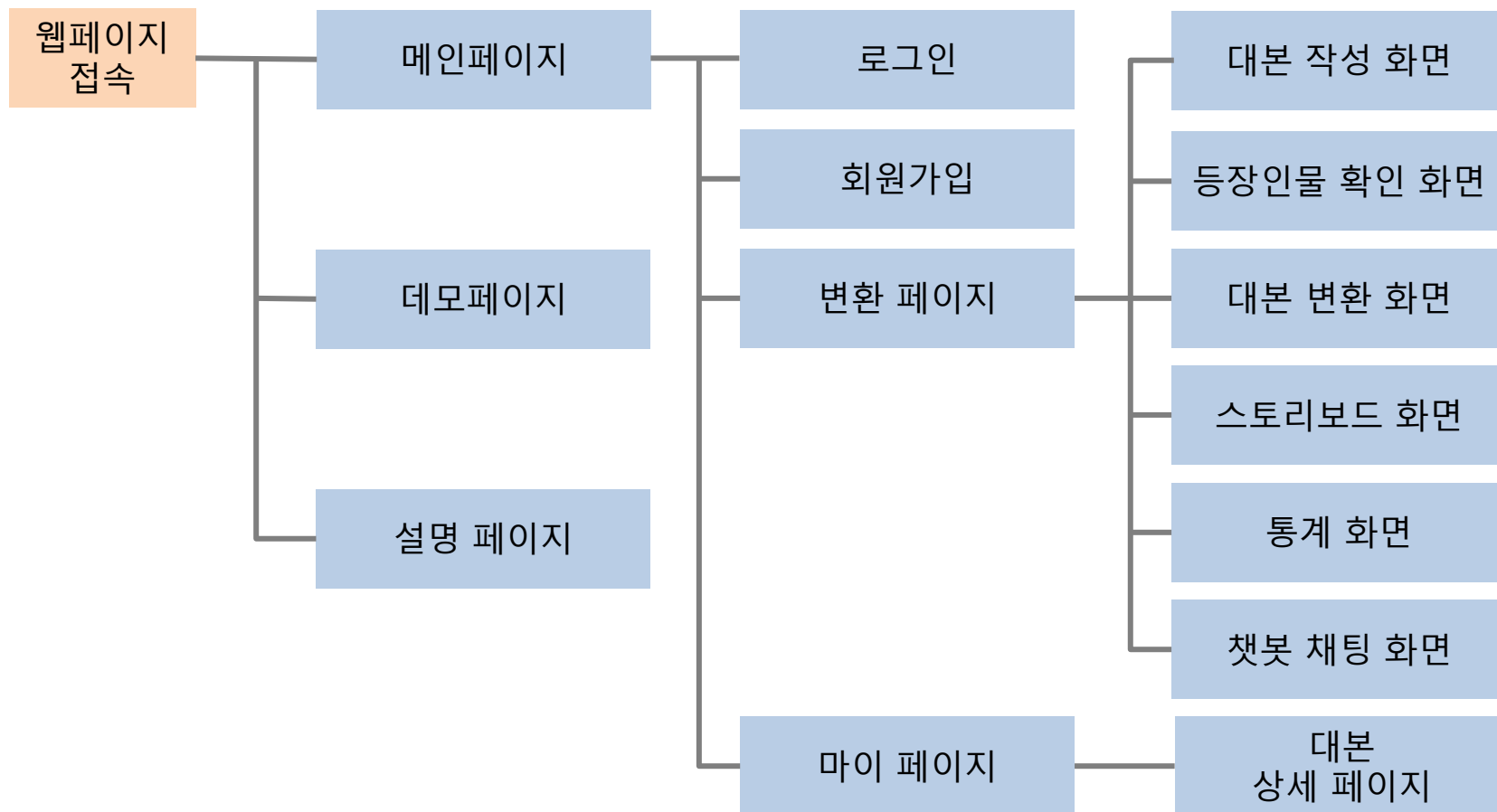
Flask Server

- 대본(ETRI API, spacy model), 스토리보드(Diffusion Model) 데이터 생성 및 반환
- 사용자 채팅 대한 챗봇(GPT-3.5) 채팅 결과 반환

Deployment

- 프로젝트 배포: NginX, AWS EC2. vast.ai
- 메타데이터 저장: S3에 저장 후 URL을 DB에 저장
- 데이터베이스 운영 및 배포: RDS 사용

| 메뉴 구성도

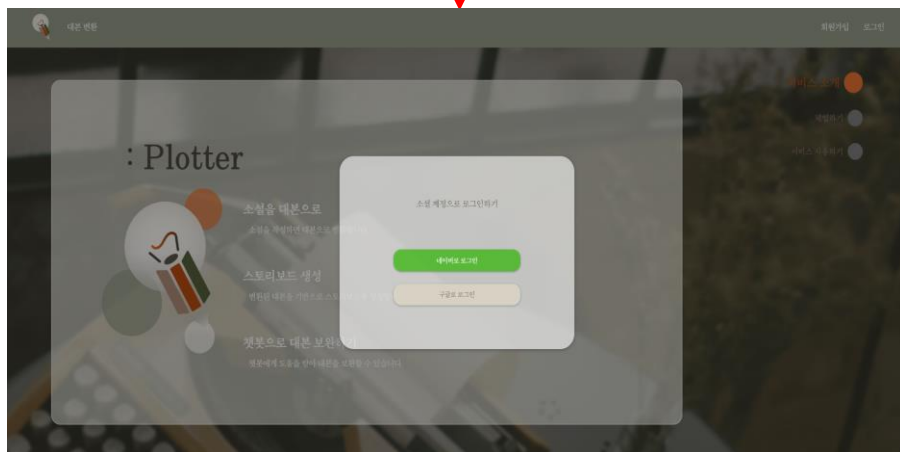


| 화면설계서 - 사용자 인터페이스(sw)

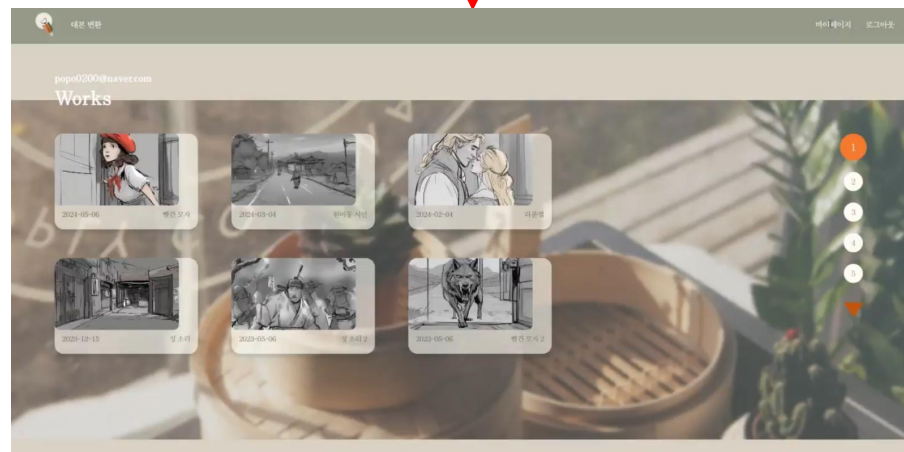
1. 나브바 동작



[로그인 모달]



[마이페이지]



| 화면설계서 - 사용자 인터페이스(SW)

2. 메인페이지

[메인페이지 - 서비스 소개]



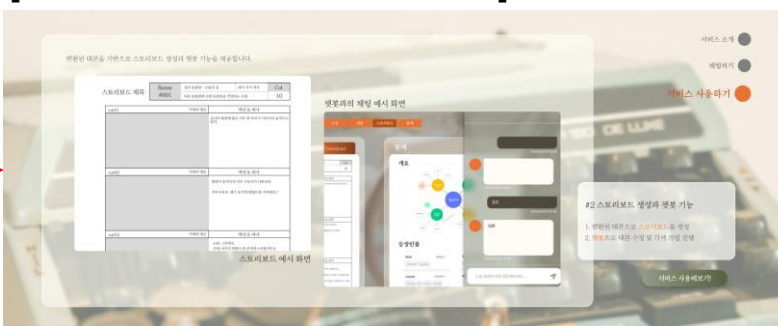
[메인페이지 인디케이터]



[메인페이지 - 소설의 대본화 데모 체험]



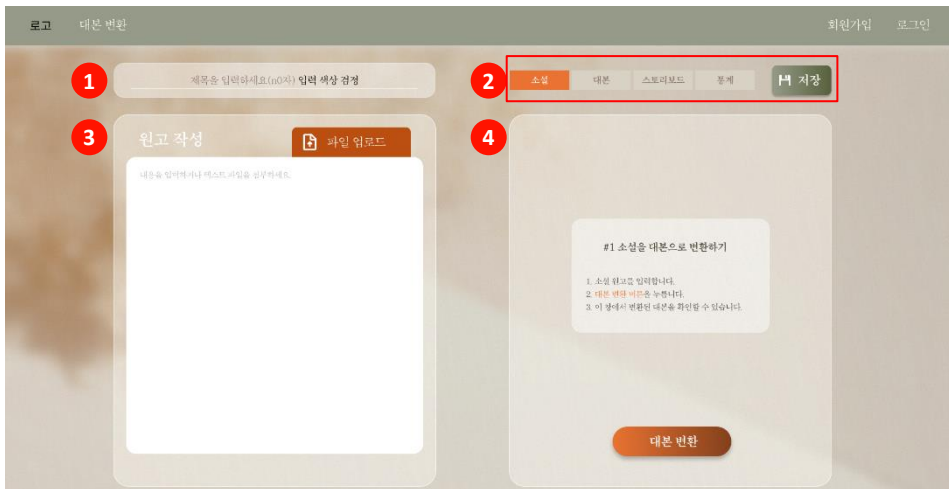
[메인페이지 - 결과물 예시 페이지]



| 화면설계서 - 사용자 인터페이스(sw)

2. 변환 페이지

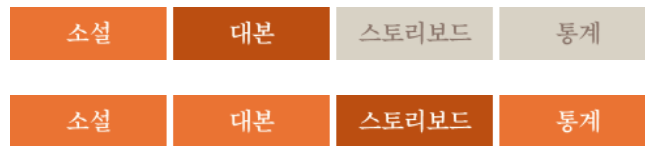
[초기 화면]



- ① 제목 입력창
- ② 인디케이터, 저장 버튼
- ③ 변환 컴포넌트 (활성화 상태)
- ④ 변환 컴포넌트 (튜토리얼 상태)

[인디케이터 동작]

- 단계 활성화 시 회색→주황색
- 사용자가 보고 있는 컴포넌트는 어두운 주황색



[모든 컴포넌트 활성화 시 화면]

- 가로로 긴 웹페이지를 2단 레이아웃 표시, 인디케이터로 이동

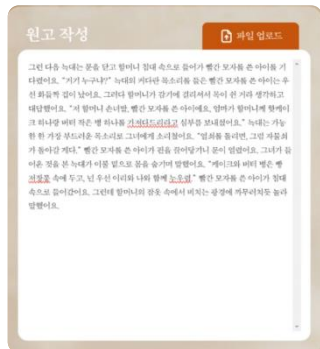


| 화면설계서 - 사용자 인터페이스(sw)

2. 변환 페이지

- 각 컴포넌트의 초기 화면은 튜토리얼 표시
- 진행 버튼 클릭 시 단계적으로 활성화

[소설 원고 입력]



[등장인물 확인]



표시

나타남

[대본화]



표시

나타남

[스토리보드]



표시

나타남

[통계]

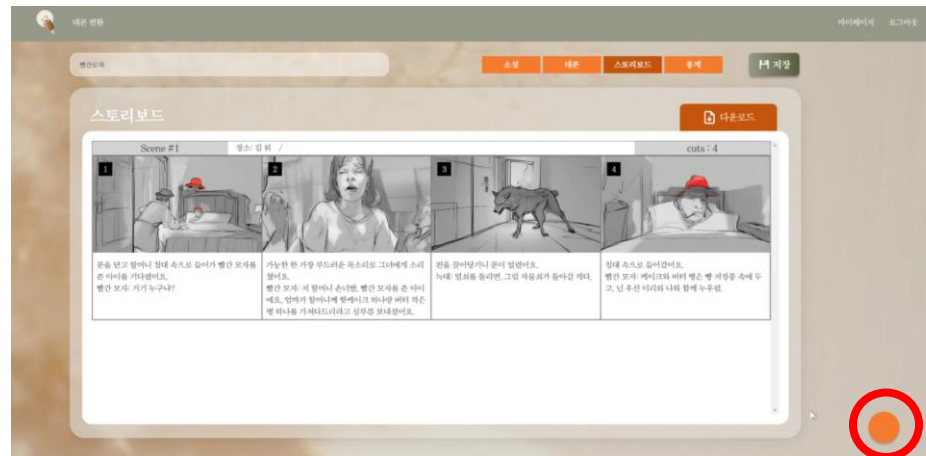


| 화면설계서 - 사용자 인터페이스(sw)

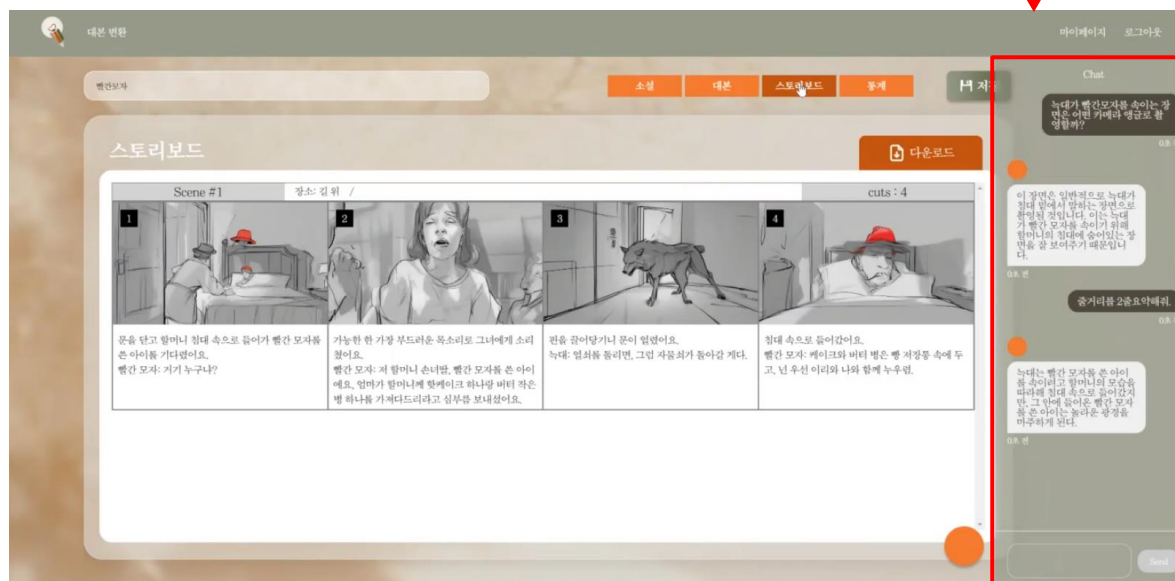
2. 변환 페이지: 챗봇 컴포넌트

[챗봇 대화창]

- 대본으로 변환한 후에 플로팅 버튼이 보임
- 플로팅 버튼 클릭 시 드로어레이아웃 펼쳐짐
- 뒤로가기 버튼 클릭 시 드로어레이아웃 접힘



버튼 클릭

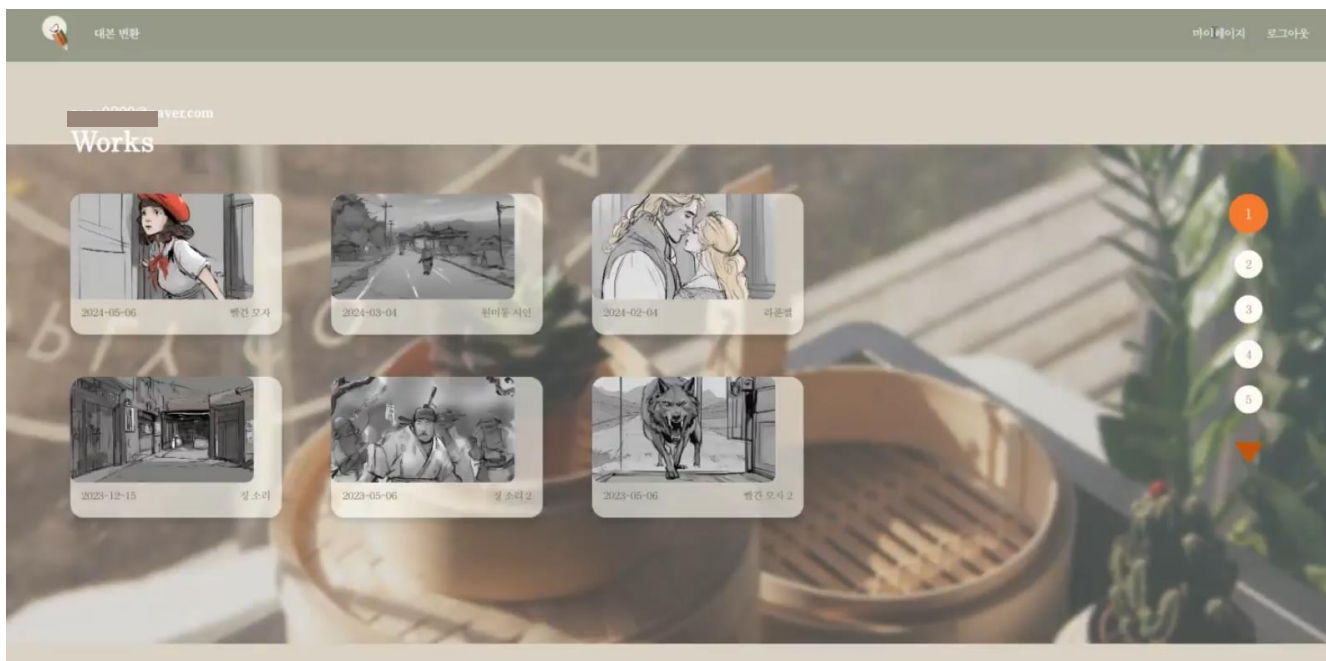


| 화면설계서 - 사용자 인터페이스(sw)

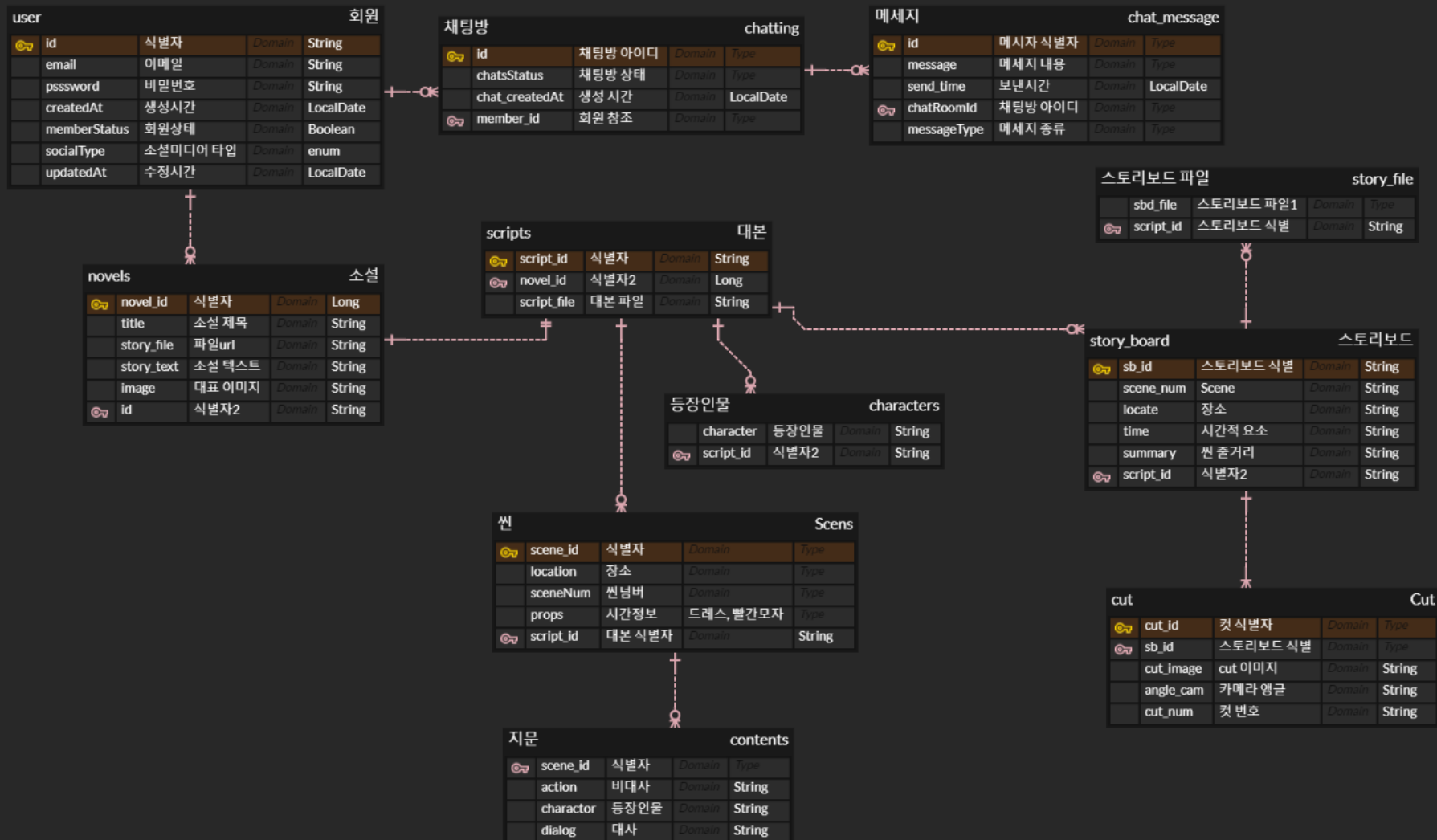
3. 마이페이지

[마이페이지]

- 사용자 정보(이메일) 확인
- 사용자가 지금까지 생성한 대본, 스토리보드 열람



| 엔티티관계도 - ERD



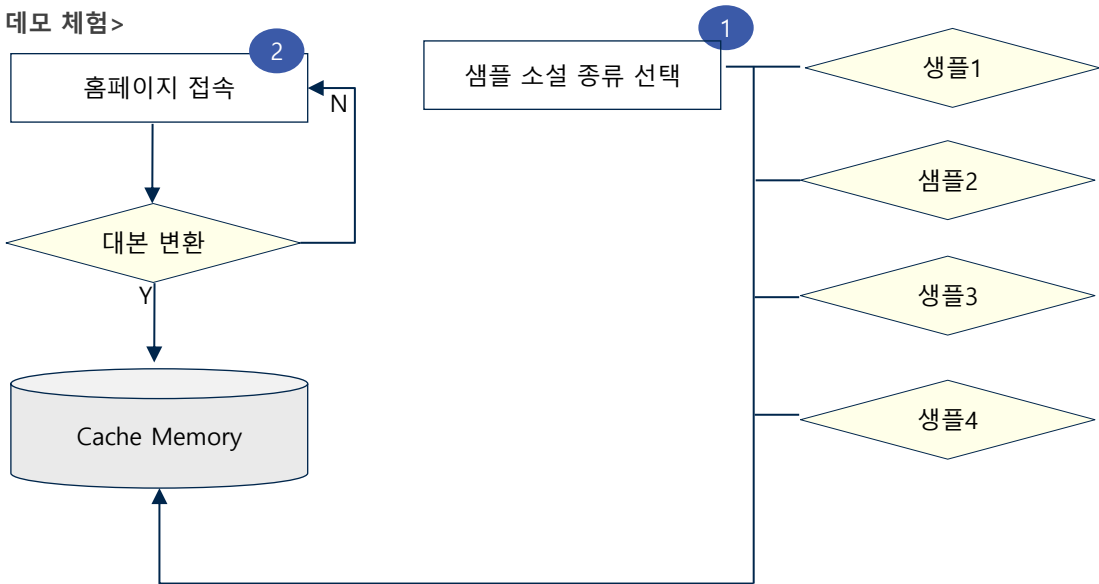
| 기능 처리도(기능 흐름도)

실무 산출물 형식

프로그램 ID	ALL-01-02	프로그램 명	NLP 기반 소설 IP 확장 솔루션	작성일	2024.07.09	Page	1/6
개요	서비스를 본격적으로 사용하기 전, 사용자는 우리 서비스에서 제공하는 기능 중 하나인 '소설의 대본화 기능'을 체험					작성자	김혜수

기능 흐름도

<서비스 데모 체험>



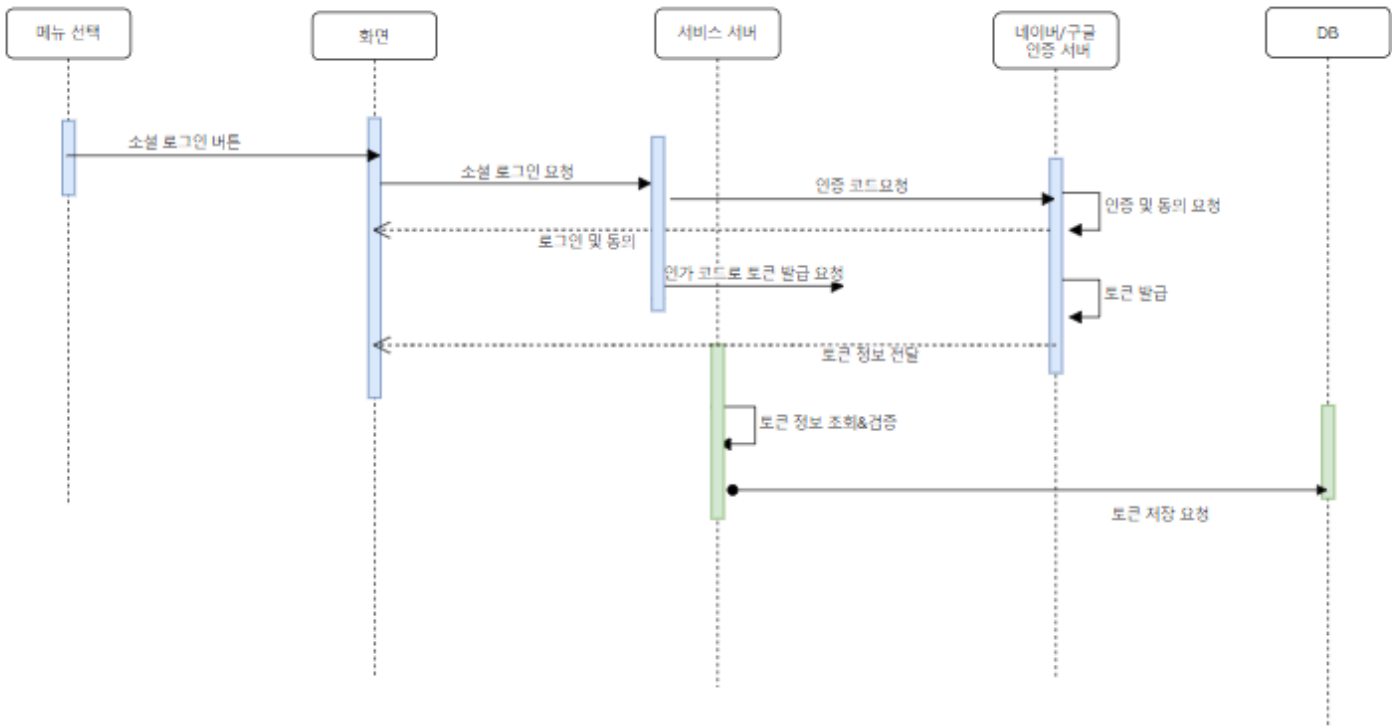
- 1 사용자는 샘플 소설을 선택할 수 있다.
- 2 주어진 샘플 4가지의 대본 변환 결과를 확인할 수 있다.

| 기능 처리도(기능 흐름도)

실무 산출물 형식

프로그램 ID	LOG-01-01, LOG-01-02 LOG-01-03	프로그램 명	NLP 기반 소설 IP 확장 솔루션	작성일	2024.07.09	Page	2/6
개요	Google과 Naver 계정을 통해 사용자 인증 작업 진행(로그인)					작성자	김혜수

기능 흐름도

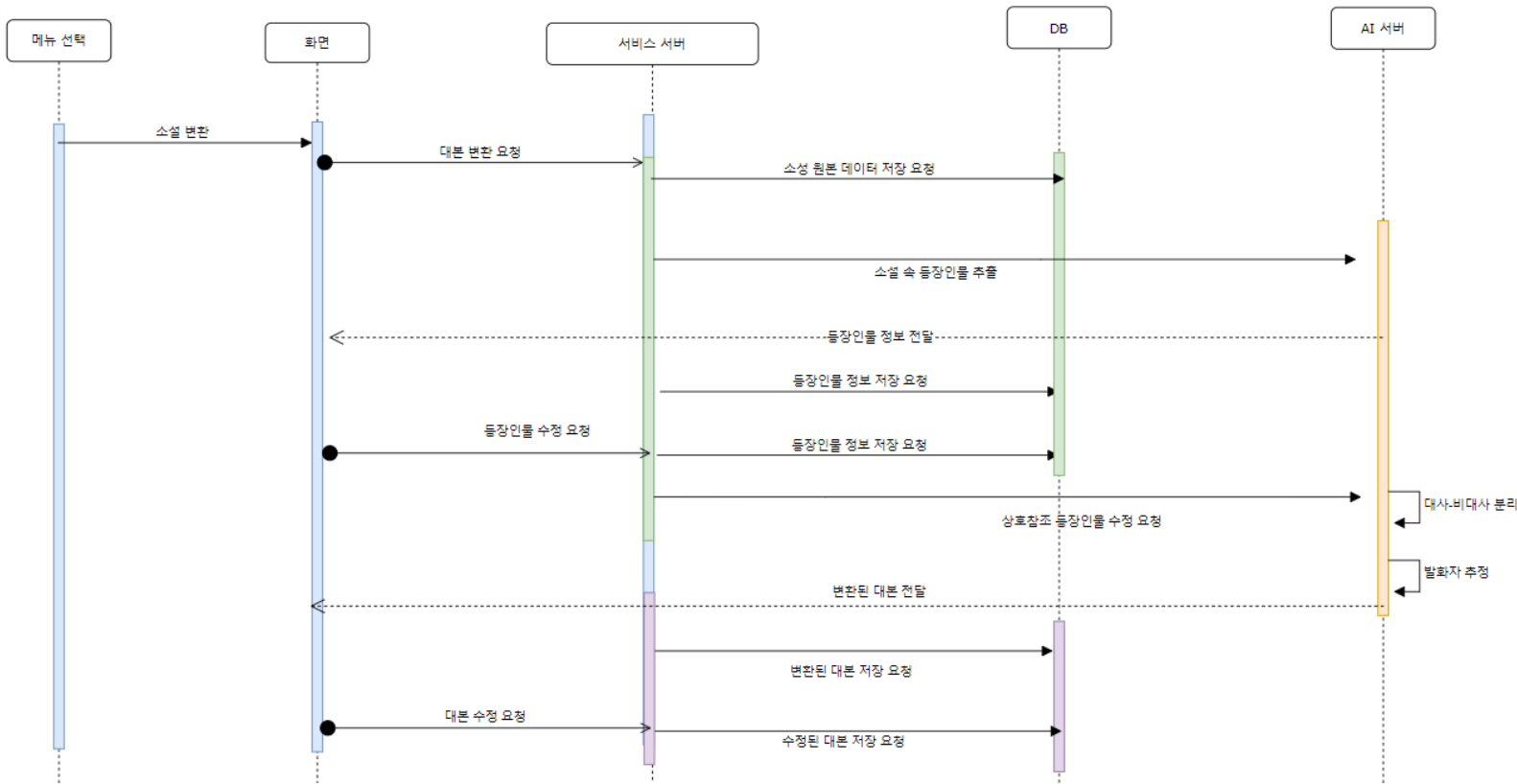


| 기능 처리도(기능 흐름도)

실무 산출물 형식

프로그램 ID	SCR-01-01, SCR-02-01 SCR-02-02	프로그램 명	NLP 기반 소설 IP 확장 솔루션	작성일	2024.07.09	Page	3/6
개요	소설을 대본으로 변환하는 작업 진행(등장인물 파악, 대사와 비대사 분리, 발화자 추정 작업 진행)					작성자	김혜수

기능 흐름도

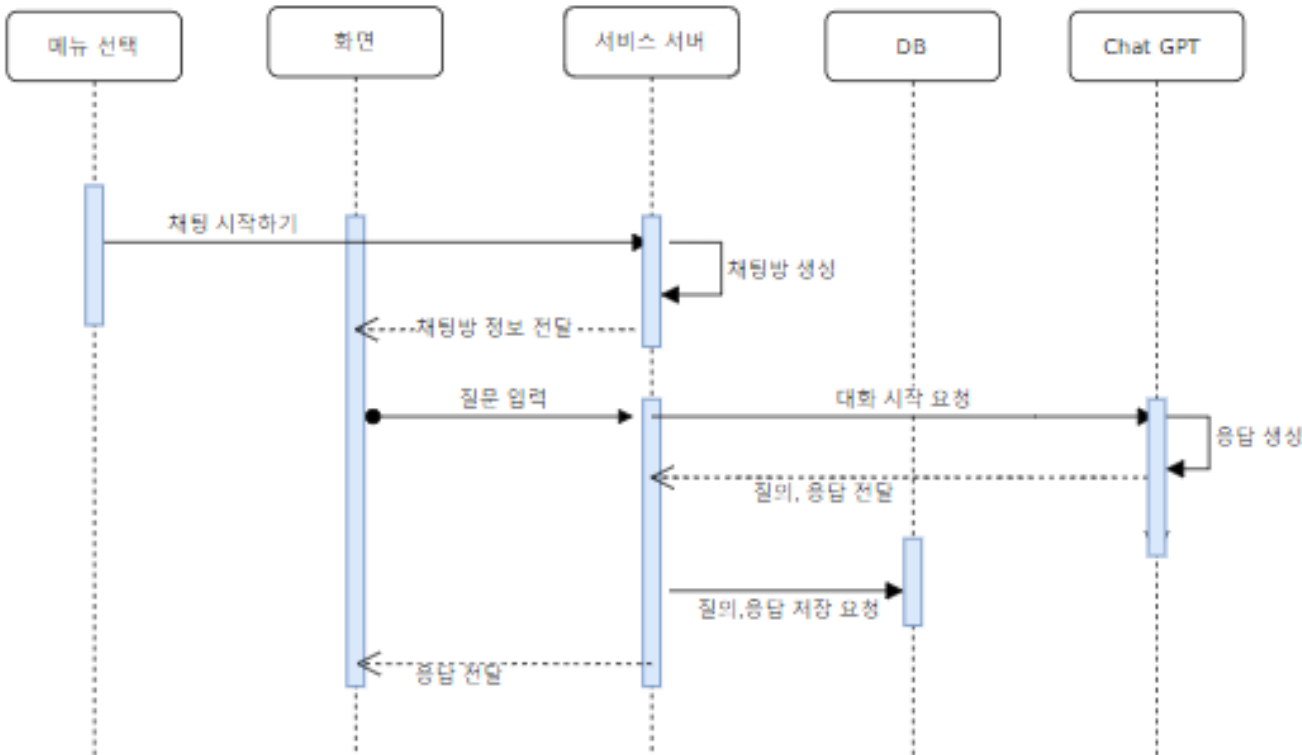


| 기능 처리도(기능 흐름도)

실무 산출물 형식

프로그램 ID	CHT-01-01, CHT-01-02	프로그램 명	NLP 기반 소셜 IP 확장 솔루션	작성일	2024.07.09	Page	4/6
개요	GPT-3.5를 사용하여 프롬프트 사용하여 질의응답 채팅 활성화 과정					작성자	김혜수

기능 흐름도

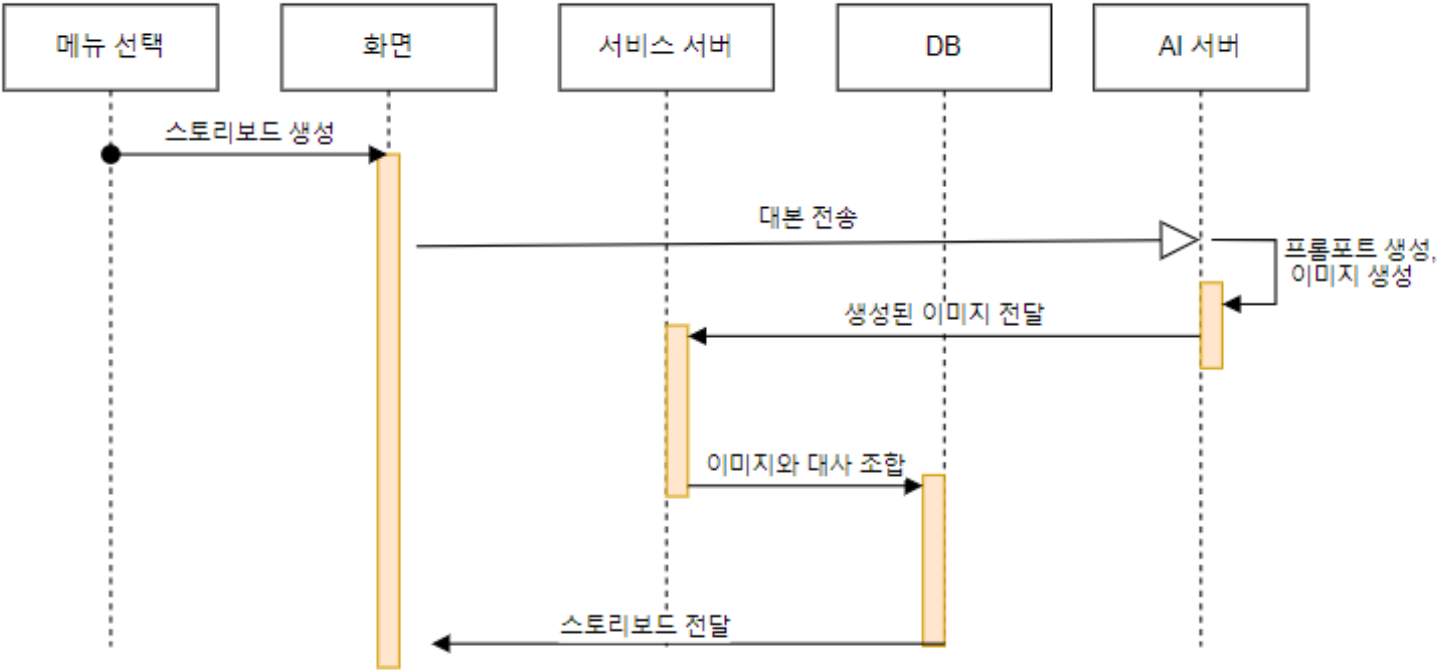


| 기능 처리도(기능 흐름도)

실무 산출물 형식

프로그램 ID	MYP-01-01	프로그램 명	NLP 기반 소설 IP 확장 솔루션	작성일	2024.07.09	Page	5/6
개요	대본을 스토리보드로 변환하는 작업(스토리보드 생성, 저장)					작성자	김혜수

기능 흐름도

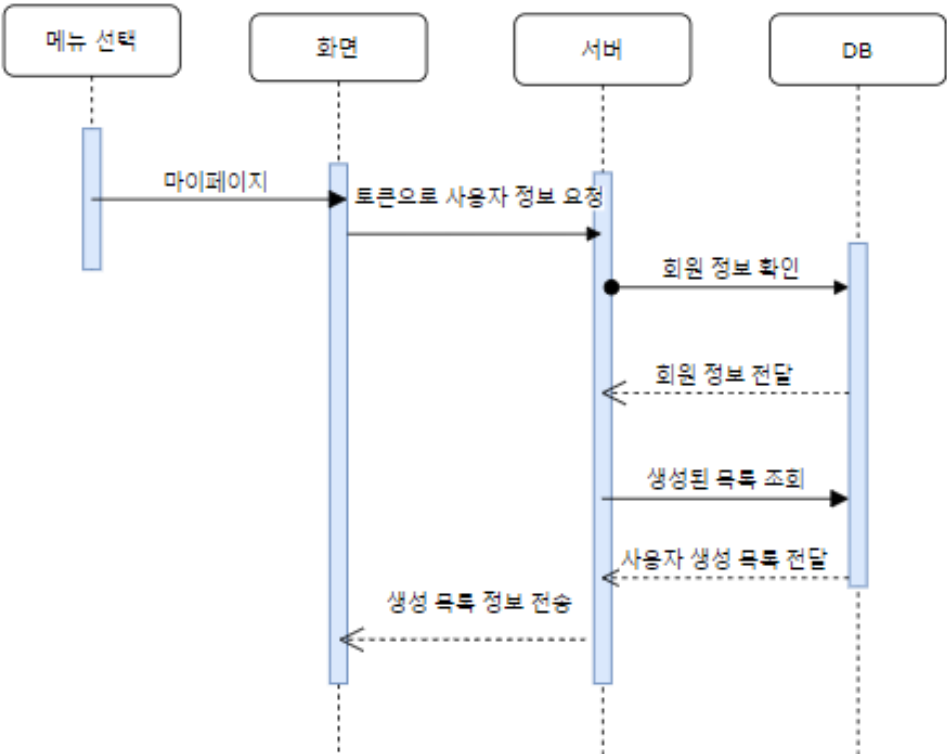


| 기능 처리도(기능 흐름도)

실무 산출물 형식

프로그램 ID	MYP-01-01	프로그램 명	NLP 기반 소설 IP 확장 솔루션	작성일	2024.07.09	Page	6/6
개요	대본 및 스토리보드 히스토리를 마이페이지에서 확인					작성자	김혜수

기능 흐름도



| 프로그램 - 목록

기능 분류	기능번호	기능 명
LOG	LOG-01-01	회원가입(구글 연동)
	LOG-01-02	회원가입(네이버 연동)
	LOG-01-03	로그인
NOV	NOV-01-01	소설 텍스트 작성
	NOV-01-02	소설 파일로 업로드
SCR	SCR-01-01	대본 변환 요청
	SCR-02-01	대본 수정
	SCR-02-02	주인공 수정
	SCR-03-01	대본 파일 다운로드
SBD	SBD-01-01	스토리 보드 생성 요청
ALL	ALL-01-01	서비스 소개
	ALL-01-02	기능 체험하기
CHT	CHT-01-01	채팅방 생성하기
	CHT-01-02	채팅 메시지 보내기
STA	STA-01-01	통계자료 시각화 이미지 확인하기
MYP	MYP-01-01	변환된 대본과 스토리 보드 확인하기

| 테이블 정의서 - ERD

<소설>

키	논리	설명	타입	Null 허용
PK	novel_id	식별자	Long	N
	title	소설 제목	String	N
	story_file	파일url	String	Y
	story_text	소설 텍스트	String	N
	image	대표 이미지	String	Y
FK	id	식별자2	String	N

<대본>

키	논리	설명	타입	Null 허용
PK	script_id	식별자	Long	N
FK	story_id	스토리보드 식별자	Long	N
	script_file	대본 파일	String	Y

등장인물

키	논리	설명	타입	Null 허용
	charater	등장인물	String	N
FK	script_id	대본 식별	Long	N
content				
키	논리	설명	타입	Null 허용
	scene_id	씬 식별자	Long	N
	action	대본 식별	String	N
	dialog	대사	String	N
	character	등장인물	String	N

| 테이블 정의서 - ERD

<회원>

키	논리	설명	타입	Null 허용	값 목록
PK	id	식별자	String	N	
	email	이메일	String	Y	
	psssword	비밀번호	String	Y	
	createdAt	생성시간	LocalDate	Y	
	memberStatus	회원상태	Boolean	Y	
	socialType	소셜미디어 타입	enum	Y	NAVER,GOOGLE
	token	토큰	String	Y	
	updatedAt	수정시간	LocalDate	Y	

<채팅>

키	논리	설명	타입	Null 허용	값 목록
PK	id	채팅방 아이디	String	N	
	chatsStatus	채팅방 상태	ENUM	Y	ACTIVE,INACTIVE
	chat_createdAt	생성 시간	LocalDate	Y	
FK	member_id	회원 참조	String	N	
FK	story_id	소셜 식별자	Long	N	

<메세지>

키	논리	설명	타입	Null 허용	값 목록
PK	id	메시자 식별자	Long	N	
	message	메세지 내용	String	Y	
	send_time	보낸시간	LocalDate	Y	
FK	chatting_id	채팅방 아이디	String	N	

| 테이블 정의서 - ERD

<스토리보드>

키	논리	설명	타입	Null 허용
PK	sb_id	스토리보드 식별	Long	N
	scene_num	Scene	String	Y
	locate	장소	String	Y
	time	시간적 요소	String	Y
	summary	썸 줄거리	String	Y
FK	script_id	식별자2	String	N

<스토리보드 정보>

키	논리	설명	타입	Null 허용
PK	sf_id	식별자	Long	N
	Field	액션&대사	String	N

<컷>

키	논리	설명	타입	Null 허용
PK	cut_id	컷 식별자	Long	N
FK	sb_id	스토리보드 식별	Long	N
	cut_image	cut 이미지	String	Y
	angle_cam	카메라 앵글	String	Y
	cut_num	컷 번호	String	Y

| 핵심소스코드

1. 등장인물 상호참조해결: (1) ETRI API 호출 및 명사구 그룹핑

소셜 데이터를 받고 동일한 객체를 가리키는 명사구들을 그룹핑하여 상호참조해결을 수행한다. ETRI 상호참조해결 API를 사용하여 이를 구현했다. **etri_request()** 함수에서 API를 사용하기 위해 데이터(text)와 인증키를 request body로 넣는다. Urllib3 라이브러리를 사용하여 POST API를 호출하고 json 형식으로 결과를 받는다.

반환값의 형식은 객체이지만, 그 중에서도 그룹핑된 명사구들의 배열을 사용한다. **mention_grouping()** 함수에서 그룹핑된 멘션들에서 필요한 정보만을 배열에 저장하였다.

```
def etri_request(text):
    openApiURL = "http://aiopen.etri.re.kr:8000/Coreference"
    accessKey = os.getenv("ETRI_AUTH_KEY")

    # request
    requestJson = {
        "argument": {
            "text": text
        }
    }

    # response
    http = urllib3.PoolManager()
    response = http.request(
        "POST",
        openApiURL,
        headers={"Content-Type": "application/json; charset=UTF-8", "Authorization": accessKey},
        body=json.dumps(requestJson)
    )

    json_data = str(response.data, "utf-8")
    parsed_data = json.loads(json_data)
    value = parsed_data['return_object']

    return value
```

<코드> ETRI API를 호출하는 etri_request 함수

```
def mention_grouping(mentions):
    # 동일 멘션 그룹핑
    group_mention = []
    group_mentions = []

    print("mentions:", mentions)
    for ment in mentions:
        for item in ment:
            group_mention.append(item['text_short'])
            group_mentions.append(group_mention)
            group_mention = []

    print("group_mentions: ", group_mentions)

    return group_mentions
```

<코드> 멘션들을 그룹핑을 하는
mention_grouping 함수

| 핵심소스코드

1. 등장인물 상호참조해결: (2) 중심어 선정

그룹핑한 명사구들의 배열을 갖고 각 그룹(각 등장인물)을 대표할 중심어를 선정한다. **filter_proppernouns()** 함수에서 Okt 라이브러리를 사용하여 품사태깅을 진행한다. 대명사에 해당하는 명사구는 제외하고, 명사구에 붙어 있는 조사는 제거하여 **filtered_word** 변수에 저장한다. Okt 라이브러리를 사용하여 대명사의 추출이 잘 되지 않는 경우가 있어, 대명사 사전 pronouns를 생성했다. 각 등장인물별로 등장인물을 지칭하는 명사들이 그룹핑되어 있는 2차원 배열을 얻을 수 있다. 얻은 배열을 바탕으로 가장 많이 등장하는 명사를 찾는다. Counter을 사용하여 등장하는 명사의 수를 세어 가장 많이 등장한 명사를 추출하여 이를 center_character 배열에 저장하였다. **center_character**은 각 등장인물에 대한 중심어(대표 명사구)를 저장한다.

```
def filter_proppernouns(group_mentions):
    ## 그룹 별로 대체할 고유명사 확정하기 ##
    global pronouns

    # 품사태깅 및 분류, 명사만 뽑기
    okt = Okt()
    tagging_total_words = []
    tagging_group_words = []

    for group in group_mentions:
        for word in group:
            # 품사 태깅
            tagged_words = okt.pos(word)
            # 조사, 대명사 제거 후 어절 재구성
            filtered_word = ''.join([morph for morph, tag in tagged_words if tag != 'Josa'
                                     and (tag != 'Noun' or morph not in pronouns)])

            if filtered_word != '':
                tagging_group_words.append(filtered_word)
            tagging_total_words.append(tagging_group_words)
            tagging_group_words = []

    print("tagging_total_words: ", tagging_total_words)
```

```
# 고유명사 확정하기
center_character = []
for group in tagging_total_words:
    counter = Counter(group)

    # 가장 많이 겹치는 문자열 찾기
    # 만약에 겹치는 문자가 없으면 문장에서 먼저 출현한 고유명사를 배열에 추가.
    most_common_string = counter.most_common(1)[0][0]

    print(most_common_string)
    center_character.append(most_common_string)
print("center_character pre:", center_character)
```

```
# dic #
pronouns = {
    '나', '나를', '내가', '내', '나의', '나에게',
    '너', '너를', '네가', '네', '너의', '너에게',
    '그', '그를', '그가', '그의', '그에게',
    '그녀', '그녀를', '그녀가', '그녀의', '그녀에게',
    '우리', '우리를', '우리가', '우리의', '우리에게',
    '당신', '당신을', '당신이', '당신의', '당신에게',
    '자신', '자신을', '자신이', '자신의', '자신에게',
    '그들', '그들을', '그들이', '그들의', '그들에게',
    '그녀들', '그녀들을', '그녀들이', '그녀들의', '그녀들에게'}
```

<코드> 그룹별 대체할 고유명사(중심어) 확정

| 핵심소스코드

1. 등장인물 상호참조해결: (3) 대명사 추출 및 중심으로 치환

```
# 개체 타입 추출 알고리즘
def filter_NE(value):
    # ne_character_type = 등장인물로 추출할 타입(직위/직책, 동물, 사람이름)

    # 알고리즘 고안 중.. 중심어의 타입 확인, 타입에 포함되지 않으면 상호참조해결 항목에서 제거
    ne = [item for sentence in value['sentence'] for item in sentence['NE']]
    print("ne: ", ne)

    ne_character_type = ["CV_POSITION", "CV_RELATION", "AM_OTHERS ", "AM_INSECT", "AM_BIRD", "AM_FISH", "AM_MAMMALIA", "AM_AMPHIBIA",
                        "AM_REPTILIA", "PS_NAME", "AFW_DOCUMENT"]
    ne_characters = [item['text'] for item in ne if item['type'] in ne_character_type]
    filter_ne = list(set(ne_characters))
    print("ne_characters: ", filter_ne)

    return filter_ne
```

```
def confusion_resolution(center_character, mentions, total_words, filter_noun):
    global pronouns
    okt = Okt()
    #print(mentions)
    for i in range(0, len(mentions)):
        for j in range(0, len(mentions[i])):
            sent_id = mentions[i][j]['sent_id']_# 해당 멘션의 문장 인덱스
            start_eid = mentions[i][j]['start_eid']_# 해당 멘션의 문장 내 시작 인덱스
            check_words = total_words[sent_id][start_eid]_# 어절 단위 데이터, 해당 멘션의 위치
            if check_words in pronouns: # 대명사가 포함되었을 때
                tagged_words = okt.pos(check_words)
                #print("tagged_words: ", tagged_words)

                # 조사, 대명사 제거 후 어절 재구성
                josa = [morph for morph, tag in tagged_words if tag == 'Josa']
                if len(josa) == 0:
                    filtered_word = center_character[i]
                else:
                    filtered_word = center_character[i] + [morph for morph, tag in tagged_words if tag == 'Josa'][0]
                #print("filtered_word: ", filtered_word)
                # 삼입하기 + 추출한 조사와 합하기
                total_words[sent_id][start_eid] = filtered_word
                #print("word[sent_id][start_eid]: ", total_words[sent_id][start_eid])

    # filter_noun에 포함하는지 확인, 포함하는 리스트에서 제거
    new_center_character = [item for item in center_character if any(ne_item in item for ne_item in filter_noun)]
    print("center_character new:", new_center_character)

    return total_words, new_center_character
```

추출한 중심어들 중, 태그에 해당하는 않는 명사구들은 등장인물로 등장할 가능성이 없다고 판단하여 필터링을 진행했다.

태그는 ETRI API에서 추출된 개체명 인식을 기반으로 하며, 지위, 관계, 동물, 사람 이름으로 선정했고, 이 태그 내의 범위에 해당하는 중심어를 filter_ne에 저장하였다.

소설 텍스트 내에서 대명사에 해당하는 명사구들을 등장인물의 중심으로 치환한다.

Okt를 사용하여 품사태깅을 진행한 후, 대명사인 경우에 해당 명사구가 속한 그룹(등장인물)의 중심으로 치환한다. **mentions**에는 해당 명사구의 인덱스에 대한 문장의 인덱스, 문장 내에서 등장하는 명사구의 인덱스 정보가 저장되어 있어, mention에서 sent_id, start_id를 사용하여 명사의 위치에 중심으로 대체한다. 만약 대명사+조사 형태일 경우, 대명사와 조사를 slice하고 대명사만 치환한 후에 다시 조사와 합친다.

<코드> 대명사를 추출하여 중심으로 치환

| 핵심소스코드

2. 비대사에서 행동지문 추출하기 : 품사 태깅을 기반으로 진행

```
def find_sentences_with_specific_tags(narration, http, open_api_url, headers):
    sentences_with_specific_tags_and_last_word_VV = []
    processed_sentences = set() # Set to store processed sentences

    for sentence in narration:
        if sentence in processed_sentences:
            continue # Skip already processed sentences

        result = analyze_sentence(sentence, http, open_api_url, headers)
        morphemes = result['return_object']['sentence'][0]['morph']

        # 형태소 중에 'NP'와 'JX' 또는 'NNP'와 'JX'를 모두 포함하는지 확인
        has_np_jx = any(m['type'] == 'NP' for m in morphemes) and any(m['type'] == 'JX' for m in morphemes)
        has_nnp_jx = any(m['type'] == 'NNP' for m in morphemes) and any(m['type'] == 'JX' for m in morphemes)
        has_nng_jx = any(m['type'] == 'NNG' for m in morphemes) and any(m['type'] == 'JX' for m in morphemes)
        has_np_jks = any(m['type'] == 'NP' for m in morphemes) and any(m['type'] == 'JKS' for m in morphemes)
        has_nnp_jks = any(m['type'] == 'NNP' for m in morphemes) and any(m['type'] == 'JKS' for m in morphemes)
        has_nng_jks = any(m['type'] == 'NNG' for m in morphemes) and any(m['type'] == 'JKS' for m in morphemes)

        last_word_VV = any(
            morphemes[-1]['type'] == 'VV' for i in range(1, min(len(morphemes), 5))
        ) if morphemes else False

        # 'ef'와 'jx'가 연속으로 나오는 경우를 확인하여 제외
        has_ef_jx_sequence = any(
            morphemes[i]['type'] == 'EF' and morphemes[i + 1]['type'] == 'JX'
            for i in range(len(morphemes) - 1)
        ) if len(morphemes) >= 2 else False

        if (
            has_np_jx or has_nnp_jx or has_nng_jx or has_np_jks or has_nnp_jks or has_nng_jks and last_word_VV and not has_ef_jx_sequence:
            # 주어진 태그 부분 삭제
            start_idx, end_idx = find_np_nnp_nng_jx_jks_indices(morphemes)
            if start_idx is not None and end_idx is not None:
                remaining_morphemes = morphemes[end_idx:]
                remaining_sentence = ''.join(m['lemma'] for m in remaining_morphemes)
                remaining_sentence = sentence[sentence.find(morphemes[end_idx]['lemma']):] # 원본 형태 유지
                sentences_with_specific_tags_and_last_word_VV.append(remaining_sentence)
                processed_sentences.add(sentence)

    return sentences_with_specific_tags_and_last_word_VV
```

품사 태깅으로 동사를 구분한 뒤, 이 중에서도 인물의 행위만을 추출하기 위해 ETRI 언어분석 API를 사용하여 동사구가 주체를 나타내는 형태소를 모두 포함하는지 확인했다. 이를 대본에서 나타나는 형식에 맞춰 인물의 동작, 표정, 말투 등장시기 등을 지시하는 지문으로 변환한다.

1. 문장을 분석해 대명사, 고유명사, 일반명사 중 하나와 보조사, 주격조사중 하나를 동시에 가진 문장을 분류한다.
2. 마지막 형태소가 동사인지 확인한다.
3. 형태소 리스트에서 어말어미와 보조사가 연속적으로 나오는지 확인한다.
4. 해당 문장에서 특정 형태소 조합을 제거하고 나머지 형태소로 새로운 문장을 구성한다.
5. 추출한 문장들을 (행위지문) 형태로 나타낸다.

| 핵심소스코드

3. Spacy NER(명명된 개체명 인식)을 통한 씬 구분 및 장소,소품 분석

```

# 문장 경계 인식을 위해 sentencizer 추가
if 'sentencizer' not in nlp.pipe_names:
    nlp.add_pipe('sentencizer', before='ner')

# 문서 분석
doc = nlp(text)

# 엔티티에서 props 및 location 레이블 추출 및 저장할 사전
ner_data = {}
segments = []
combined_props = set()
current_location = None
sentence_number = 1 # 'senNum'을 위한 문장 번호
combined_sentences = [] # 동일 location에 해당하는 문장을 하나로 결합

for sentence in doc.sents:
    # 문장 내에서 location과 props 엔티티 추출
    sentence_locations = {entity.text for entity in sentence.ents if entity.label_ == 'location'}
    sentence_props = {entity.text for entity in sentence.ents if entity.label_ == 'props'}

    print(f"sentence_locations: {sentence_locations}") # 디버깅용
    print(f"sentence_props: {sentence_props}") # 디버깅용

    # 새로운 location이 등장할 경우 senNum을 증가시키고, 이전 장소 정보 저장
    if sentence_locations:
        # 현재 location이 있다면 문장을 저장
        if current_location is not None:
            # 저장
            ner_data[sentence_number] = {
                "location": current_location,
                "props": list(combined_props)
            }
            segments.append((sentence_number, " ".join(combined_sentences)))

        # senNum 증가
        sentence_number += 1

```

Spacy는 텍스트를 처리하기 위해 여러 구성 요소가 순차적으로 적용되는 파이프라인을 사용한다. 일반적인 파이프라인은 토큰라이저(tokenizer), 태거(tagger), 파서(parser), NER(명명된 개체 인식기), 텍스트 속성 분석기 등의 모듈로 이루어져 있는데, sentencizer로 문장 경계를 인식하기 위한 파이프라인 구성 요소로 NER(명명된 개체 인식) 이전에 문장을 분리했다.

| 핵심소스코드

4. CSS(문맥적 의미 유사성) 점수 산정기준 발화자 매칭

```
class CandidateScoringNetwork(nn.Module):
    def __init__(self, model_name='bert-base-uncased'):
        super(CandidateScoringNetwork, self).__init__()
        self.bert = BertModel.from_pretrained(model_name)
        self.fc = nn.Linear(self.bert.config.hidden_size, 1)

    def forward(self, input_ids, attention_mask):
        outputs = self.bert(input_ids, attention_mask=attention_mask)
        cls_output = outputs.last_hidden_state[:, 0, :]
        score = self.fc(cls_output)
        return score

def process_document(sentences, candidates):
    #quotes = extract_quotes(document)
    quotes = []
    results = []
    for sent in sentences:
        if sent['type'] == 0: # 비대사
            quotes.append(sent)
    #print(quotes: ", quotes)
    #for quote, original_sentence in quotes:
    for quote in quotes:
        css_segments = []
        for candidate in candidates:
            nearest_mention = find_nearest_mention(quote, sentences, [candidate])
            css_segment = compute_css(quote, nearest_mention)
            css_segments.append(css_segment)

        input_ids, attention_masks = prepare_inputs(tokenizer, css_segments)
        model = CandidateScoringNetwork('bert-base-uncased')
        model.eval()

        with torch.no_grad():
            scores = model(input_ids, attention_masks)

        best_candidate_idx = torch.argmax(scores).item()
        best_candidate = candidates[best_candidate_idx]

        quote['character'] = best_candidate
        results.append(quote)
```

문장들에서 인용문을 필터링 (sent['type'] == 0을 사용하여 대사가 아닌 문장을 식별한다. 문서 내에서 각 후보자의 가장 가까운 언급을 찾고 인용문과 해당 언급 **사이의 문맥적 의미 유사도**(CSS, Contextual Semantic Similarity)를 계산한다.

이때 CSS는 BERT의 마지막 hidden state에서 [CLS] 토큰의 표현을 추출 후 완전 연결 레이어에 통과시켜 점수를 생성한다.

| 핵심소스코드

3. WebFlux 비동기 통신

```
public Mono<MessageDTO.MessageResponseDto> sendGPTMessage(String
chatRoomId, MessageDTO.MessageRequestDto message) {
    // 채팅방 유효성 검사 및 WebClient 비동기 호출
    return Mono.fromCallable(() -> {
        return chatRoomRepository.findById(chatRoomId)
            .orElseThrow(() -> new
GeneralHandler(ErrorStatus._BAD_REQUEST));
    })
        .flatMap(chatRoom -> {

            // GPT 요청 DTO 생성
            MessageDTO.MessageGPTRequestDto messageGPTRequestDto
= MessageDTO.MessageGPTRequestDto
                .builder()
                .message(message.getMessage())

            .context(chatRoomRepository.findStoryTextByChatRoomId(chatRoomId))
                .build();

            // WebClient 비동기 호출
            return webClientService.responseGPT(messageGPTRequestDto)
                .flatMap(responseGPT -> {
                    // 메시지 엔티티 생성
                    MessageEntity messageEntity =
MessageDTO.fromDto(responseGPT, chatRoom,
MessageEntity.MessageType.GPT);

                    /// 메시지 저장 후 저장된 메시지 엔티티 반환
                    return Mono.fromCallable(() ->
messageRepository.save(messageEntity))
                        .map(MessageDTO::fromEntity); // 저장된 메시지
엔티티로부터 DTO 반환
                });
        });
}
```

스프링 부트의 **WebFlux**는 비동기 및 논블로킹 (non-blocking) 기반으로 동작하는 웹 프레임워크입니다. WebFlux는 리액티브 프로그래밍을 위한 도구로, Mono와 Flux를 활용해 데이터를 비동기적으로 처리하고 반응형 애플리케이션을 구축한다.

이벤트 루프(Event Loop) 기반의 논블로킹 방식으로 요청을 처리한다. 이 방식의 이점은 스레드가 차단되지 않고, 요청 및 응답 처리 중에 리소스가 효율적으로 사용한다.

| 핵심소스코드

4. 창작물 기반 답변 챗봇:
(AI) Server - 챗봇 개발(gpt-3.5-turbo-instruct 사용)

```
def get_completion(prompt):
    response = openai.Completion.create(
        engine="gpt-3.5-turbo-instruct",
        prompt=prompt,
        max_tokens=500,
        n=1,
        stop=None,
        temperature=0.7,
    )
    return response.choices[0].text.strip()

@app.route('/chatbot', methods=['POST'])
def ask():
    data = request.get_json()
    question = data['question']
    context = data.get('context', '옛날 옛적에')
    tone = data.get('tone', '정적이고 사용설명서 같은 말투')

    prompt = f"Context: {context}\n\nQ: {question}\nA: {tone} 말투로 답변:"
    answer = get_completion(prompt)
    return jsonify({'answer': answer})

if __name__ == '__main__':
    app.run(debug=True)
```

OpenAI의 gpt-3.5-turbo-instruct 모델을 사용해 사용자가 요청한 질문에 대해 사용자가 생성한 창작물을 바탕으로 답변을 생성하는 챗봇이다.

context, question은 유저에가 생성한 소설과 유저의 질문을 받는 변수이고, 이를 get_completion() 함수에서 prompt에 넣어 답변을 받는다.

tone을 튜닝하여 일관적이고 명확한 말투로 답변을 주도록 했으며, **temperature**을 설정하여 GPT가 생성하는 문장이 더 다양해지도록 했다.

| 핵심소스코드

4. 창작물 기반 답변 챗봇: (2) 챗봇 채팅 (웹소켓 라이브러리 Stomp)

publish(발행)/subscribe(구독) 구조

publisher가 특정 topic(채팅방)에 메시지를 보내면
해당 topic을 구독하고 있는 subscriber
모두에게 메시지가 전달되는 구조

```

37 // (stomp) connect & subscribe
38 const connectHandler = async () => {
39   if (chatRoomId !== 'none') {
40     const result = await getChatList(chatRoomId);
41     setMessages(result.allMessages);
42   }
43
44   client.current = Stomp.over(() => {
45     const sock = new WebSocket(`ws://localhost:8080/websocket`);
46     return sock;
47   });
48   client.current!.debug = function (str) {}; // console.log off
49
50   client.current.connect(
51     {
52       // request header
53     },
54     () => {
55       // connectCallback 함수 설정
56       client.current!.subscribe(
57         `/sub/chats/${chatRoomId}`,
58         (msg: IMessage) => {
59           let newMessage: Message = JSON.parse(msg.body); // 메시지 내용 꺼내기
60           newMessage = { ...newMessage, isTyping: true };
61           setMessages((prevState) => [...prevState, newMessage]);

```

```

87 // (stomp) send(=publish)
88 const sendHandler = (message: string) => {
89   client.current!.send(
90     `/pub/chats/${chatRoomId}`,
91     {
92       // request header
93     },
94     JSON.stringify({
95       message: message.trim(),
96       messageType: 'QUESTION',
97     })
98   );
99 };

```

```

@Configuration
@EnableWebSocketMessageBroker
@RequiredArgsConstructor
public class WebSocketConfig implements
  WebSocketMessageBrokerConfigurer {
    private final ChatPreHandler chatPreHandler;
    private final ChatErrorHandler chatErrorHandler;

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        config.enableSimpleBroker("/sub");
        config.setApplicationDestinationPrefixes("/pub");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        registry.addEndpoint("/ws-stomp").setAllowedOrigins("*")
            .withSockJS(); // sock.js를 통하여 낮은 버전의 브라우저에서도
            websocket이 동작할수 있게 설정
        registry.addEndpoint("/websocket").setAllowedOriginPatterns("*");
        registry.setErrorHandler(chatErrorHandler);
    }

    @Override
    public void configureClientInboundChannel(ChannelRegistration registration) {
        registration.interceptors(chatPreHandler);
    }

```

| 핵심소스코드

5. 이미지 생성 모델 (SDXL 1.0)

```

28 def load_pipeline():
29     """
30     이미지 생성 모델 초기화, UNet 상태 사전 로드
31     :return:
32     """
33     global pipeline
34     device = torch.device('cuda')
35
36     pipeline = DiffusionPipeline.from_pretrained("stabilityai/stable-diffusion-xl-base-1.0", torch_dtype=torch.float16, use_safetensors=True, timeout=120)
37     pipeline = pipeline.to(device)
38     print("cached model local path: ", pipeline.config.name_or_path)
39     pipeline.load_lora_weights("storyboard/Storyboard_sketch.safetensors")
40
41     # CUDA 사용 가능 여부 확인
42     print("CUDA available:", torch.cuda.is_available())
43     # 사용할 GPU 인덱스 확인
44     if torch.cuda.is_available():
45         print("CUDA Device Count:", torch.cuda.device_count())
46         print("Current CUDA Device:", torch.cuda.current_device())
47         print("Device Name:", torch.cuda.get_device_name(0))
48
49     unknown
50 def text_to_image(prompt):
51     """
52     영어 prompt를 받고 image를 생성
53     :param prompt: string
54     :return: Image
55     """
56     global pipeline
57
58     image = pipeline(
59         prompt=prompt,
60         num_inference_steps=NUM_INFERENCE_STEPS,
61         cross_attention_kwargs={"scale": LORA_SCALE},
62         generator=torch.manual_seed(0)
63     ).images[0]
64
65     return image

```

huggingface에서 text to image 모델
SDXL 1.0 (Stable Diffusion XL) 로드

특정 화풍을 입히기 위해
Low-Rank Adaptation 방식으로 학습된
Unet 상태 사전(safetensors)을 적용

pipeline을 통해 프롬프트로 이미지 생성

Thank you

